

No Detectable Difference: A Blocked Factorial Experiment

Comparing Claude Fable 5 and GPT-5.6 on Proven-Solvable Coding Tasks

Drake Griffith

Actual Intelligence Labs

`christopherg@actualintelligencelabs.ai`

July 10, 2026

Abstract

We report a controlled comparison of two frontier coding agents — Claude Fable 5 (Anthropic, via the `claude` CLI) and GPT-5.6 “Sol” (OpenAI, via the `codex` CLI) — across 154 runs of a full-factorial designed experiment: model $\times$ reasoning effort $\times$ harness, blocked by task, with replicated runs treated as Bernoulli trials and seed-interleaved run order. All nine tasks were proven solvable before any model ran. Both models passed every solo run (152/152); a per-cell perfect score of 24/24 carries a 95% Wilson interval of (0.862, 1.000), and the paired head-to-head McNemar test produced zero discordant pairs — *no detectable difference*, which is a weaker and more honest claim than “equal.” Raising the reasoning-effort setting changed cost but not outcome: pass rate was 100% at every effort level while GPT-5.6 at high effort spent $\approx 80\%$ more output tokens than at low. An exact sign-flip permutation test on per-task median $\log(\text{total tokens})$ shows Claude Fable 5 materially cheaper per solved task ($p = 0.0039$), subject to a vendor-incomparability caveat on input-token accounting. Transcripts were judged blind by two independent model judges whose scores agreed within one point on 96% of 139 dual-judged runs. The headline finding is a ceiling effect: the suite was too easy to rank the models, and we say so rather than invent a winner. We publish the full design, exact statistics, power analysis, and limitations, and outline the harder-tier follow-up experiment.

1 Introduction

Most public comparisons of coding models are anecdotes: one person, one prompt, two tools, and a preference. Because large-language-model inference is non-deterministic, a single run measures what happened once, not what happens on average — and the average is the only quantity that matters when choosing which tool a team ships with. Repeat the informal comparison tomorrow and the “winner” can flip.

Structured public benchmarks have a complementary problem: frontier models now cluster near the top of the standard suites, shrinking their power to discriminate, and training-data contamination lets a model recall an answer instead of reasoning to one. Jain et al. document both effects for code generation and respond with a time-segmented benchmark of fresh contest problems [1].

We therefore ran a designed experiment rather than a leaderboard: identical tasks and conditions, enough replication to separate signal from sampling noise, exact small-sample statistics, and an explicit statement of what the sample size can and cannot detect. The design follows standard design-of-experiments practice (full factorial treatment structure, blocking, randomization) [9].

Two results follow. First, a *ceiling effect*: both models passed every solo task, so this suite establishes that both clear the tier but cannot rank them. Second, on this tier the reasoning-effort dial was pure cost: identical outcomes at every setting, materially different token bills — consistent with the “overthinking” literature on efficient reasoning [2, 3].

2 Experimental design

2.1 Treatment structure

The run matrix is a full-factorial designed experiment:

- **Factors.** Model (Claude Fable 5, GPT-5.6) $\times$ reasoning effort (2–3 levels per model) $\times$ harness (bare CLI vs. a shared agent-instruction harness).
- **Blocking.** Task (eight T1/T2 tasks plus one T3 greenfield build). Every configuration sees every task, so task difficulty cancels out of all paired comparisons.
- **Replication.** $n = 3$ repetitions per cell on T1/T2 tasks, $n = 2$ on T3. Inference is non-deterministic; a single run is an anecdote, replicated runs estimate a pass *rate*.
- **Randomization.** Run order is seed-interleaved (seed 1337) so time-of-day and API-load drift cannot systematically favor one model.

The experiment totals 154 official runs: a 120-run sweep 1 (bare, full effort ladder), a sweep 2 adding harnessed conditions and the greenfield build, and a small hybrid arm (Section 5.6).

2.2 Task suite

Nine tasks, every one *proven solvable before any model ran*:

- **Four seeded-bug tasks** with deliberately subtle defects: an off-by-one in pagination, a naive-vs-timezone-aware datetime bug, a cache key that silently drops a dimension, and an async deduplication race.
- **Four feature tickets** written in a strict six-section ticket format.
- **One greenfield CLI build** (`splitcost`, a ~ 200 -line cost-splitting tool with a reference implementation).

Each task ships a self-test: the verifier fails on the broken starting code and passes after the reference patch. All nine self-tests were green before the experiment began, so any failure is attributable to the model, not to an impossible task.

2.3 Controlled conditions and blind dual judging

Conditions were byte-identical across arms: the same harness instruction files, the same prompts, and the same deterministic `verify.sh` gate deciding pass/fail. Every transcript and diff was stripped of identifying markers and scored blind by *two* independent judges — a Claude judge and a GPT judge — on a four-dimension rubric. Dual judging is a design requirement, not decoration: LLM judges carry a documented self-preference bias [4], so a single-model judge grading its own model’s work is a conflict of interest built into the evaluator. Judge agreement is reported as a result in its own right (Section 5.7).

3 Outcome model

Each run is a Bernoulli trial: pass (`verify.sh` exit 0) or fail. A cell’s empirical pass rate $\hat{p} = k/n$ estimates its Bernoulli parameter p ; within-cell scatter reflects the Bernoulli variance $p(1 - p)$ plus model non-determinism. All per-cell sample sizes are small ($n \in \{2, \dots, 26\}$ per cell; $n = 24$ per arm in the primary sweep-1 comparisons), so every test in Section 4 is *exact* — no large-sample approximation is used anywhere except the clearly labelled power search of Section 4.4.

4 Statistical methods

All statistics are computed by a dependency-free reference implementation (`runner/stats.py`, Python standard library only) directly from the raw run and judgment logs; the implementation carries unit self-tests against known values (Appendix A).

4.1 Wilson score intervals for per-cell pass rates

For k successes in n trials with $z = z_{0.975} \approx 1.95996$, the 95% Wilson score interval [6] is

$$\frac{\hat{p} + \frac{z^2}{2n}}{1 + \frac{z^2}{n}} \pm \frac{z}{1 + \frac{z^2}{n}} \sqrt{\frac{\hat{p}(1 - \hat{p})}{n} + \frac{z^2}{4n^2}}, \quad \hat{p} = \frac{k}{n}. \quad (1)$$

Unlike the naive Wald interval $\hat{p} \pm z\sqrt{\hat{p}(1 - \hat{p})/n}$, the Wilson interval retains correct coverage at small n and at $\hat{p} \in \{0, 1\}$ — exactly the regime of this experiment, where cells post perfect scores. A perfect 24/24 is reported not as a bare “100%” but with its interval (0.862, 1.000): the data are consistent with a true failure rate anywhere from zero to $\approx 14\%$.

4.2 McNemar’s exact test for paired pass/fail comparisons

Head-to-head comparisons pair runs by task $\times$ repetition, so task difficulty cancels (the blocking pays off here). With b pairs where arm A passed and arm B failed, and c pairs the other way, only the $n_d = b + c$ *discordant* pairs carry information about a difference. The exact two-sided McNemar test [7] is a binomial test at $\frac{1}{2}$ on the discordant pairs:

$$p = \min\left(1, 2 \sum_{i=0}^{\min(b,c)} \binom{n_d}{i} \left(\frac{1}{2}\right)^{n_d}\right). \quad (2)$$

When $n_d = 0$ the test is undefined and we report “no evidence of difference” rather than a manufactured p -value.

4.3 Exact sign-flip permutation test on token cost

Token cost per solved task is compared with an exact paired permutation test [8]. For each task t we take the median of $\log(\text{total tokens})$ over *passing* runs for each model, form the paired difference $d_t = \tilde{m}_t^{\text{Fable}} - \tilde{m}_t^{\text{Sol}}$, and enumerate all 2^k sign assignments over the k tasks:

$$p = \frac{\#\{s \in \{-1, +1\}^k : |\sum_t s_t d_t| \geq |\sum_t d_t|\}}{2^k}. \quad (3)$$

The log transform makes the statistic a ratio comparison and tames the heavy right tail of token counts; medians over passing runs keep the comparison per-solved-task.

4.4 Power: the minimum detectable effect

The one deliberate approximation: the minimum detectable effect (MDE) is found by a search over the normal-approximation power of a two-sided two-proportion z -test. For per-arm size n , baseline p_1 , alternative $p_2 = p_1 + \delta$, and $\bar{p} = (p_1 + p_2)/2$,

$$\text{power}(\delta) = \Phi\left(\frac{\sqrt{n}|\delta| - z_{\alpha/2}\sqrt{2\bar{p}(1-\bar{p})}}{\sqrt{p_1(1-p_1) + p_2(1-p_2)}}\right), \quad (4)$$

and the MDE is the smallest δ with $\text{power}(\delta) \geq 0.80$ at $\alpha = 0.05$. At $n = 24$ per arm and $p_1 = 0.5$ this gives $\text{MDE} \approx 36.7$ percentage points. The reporting rule follows: *a gap smaller than the MDE gets a confidence interval, not a verdict*. We say “no detectable difference,” never “they’re equal.”

5 Results

5.1 Pass rates: both models hit the ceiling

Sweep 1 was 120/120 across both models and every effort level; folding in all solo sweep-2 runs gives **152/152** clean passes for every configuration in which a model worked alone. Table 1 reports every cell with its Wilson interval (Eq. 1); Figure 1 draws the same intervals.

model	effort	harness	pass/ n	pass rate	95% Wilson CI
Fable	medium	bare	26/26	100%	(0.871, 1.000)
Fable	medium	harness	14/14	100%	(0.785, 1.000)
Fable	high	bare	24/24	100%	(0.862, 1.000)
Hybrid	medium	harness	2/2	100%	(0.342, 1.000)
Sol	low	bare	26/26	100%	(0.871, 1.000)
Sol	low	harness	14/14	100%	(0.785, 1.000)
Sol	medium	bare	24/24	100%	(0.862, 1.000)
Sol	high	bare	24/24	100%	(0.862, 1.000)

Table 1: Per-cell pass rates with 95% Wilson score intervals. Every cell is perfect; the interval width is the honest small- n uncertainty behind each “100%.” (Hybrid: see the 2/4 disclosure in Section 5.6.)

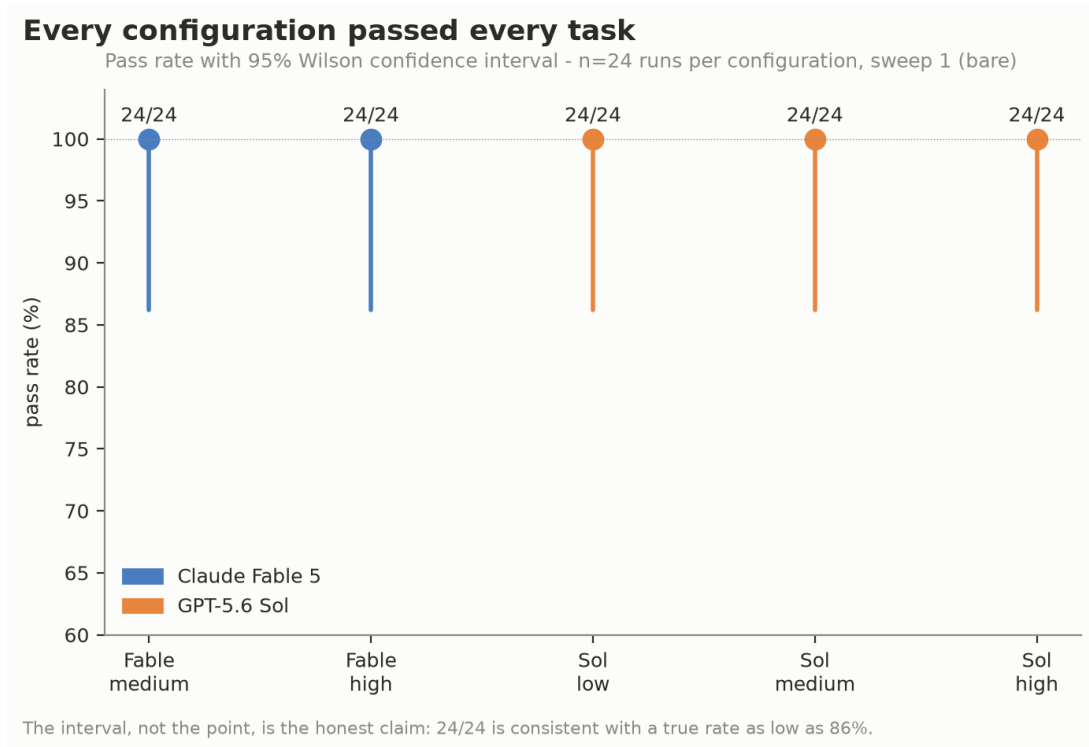


Figure 1: Pass rate by configuration with 95% Wilson intervals. Every bar sits at 100% with a lower bound set by the per-cell sample size — perfect scores drawn with their uncertainty.

Head-to-head. Each model’s best effort was selected by pass rate with a fewer-tokens tiebreak (Fable: medium; Sol: low), and the 26 task×rep pairs were compared with McNemar’s exact test (Eq. 2). All 26 pairs were pass/pass: $b = c = 0$, zero discordant pairs, nothing to test. Conclusion: **no detectable difference** between the models on this suite — a claim deliberately weaker than “equal.”

5.2 The effort dial bought nothing but tokens

Pass rate was 100% at every effort setting for both models, and every adjacent-rung McNemar comparison (Fable high vs. medium; Sol medium vs. low, high vs. medium; 24 pairs each) had zero discordant pairs. What moved was cost (Table 2, Figure 2): Claude Fable 5 at high effort used +16% output tokens vs. medium (2,008 vs. 1,736 median); GPT-5.6 at high used +80% vs. low (1,966 vs. 1,095) and $\approx 1.8\times$ the input tokens (174.7k vs. 95.2k). On tasks of this tier, “think harder” settings are pure cost — consistent with the overthinking literature: reasoning models are poor at gauging problem difficulty and elaborate on easy problems without improving correctness [2, 3].

config	pass	output tok	input tok	wall clock	turns
Fable / medium	24/24	1,736	19,412	68.2 s	7
Fable / high	24/24	2,008	19,398	58.8 s	6
Sol / low	24/24	1,095	95,248	31.9 s	1
Sol / medium	24/24	1,534	138,252	49.0 s	1
Sol / high	24/24	1,966	174,662	63.7 s	1

Table 2: Sweep-1 configurations (medians, $n = 24$ per row). Input tokens are CLI-reported and *not comparable across vendors* (Section 5.5).

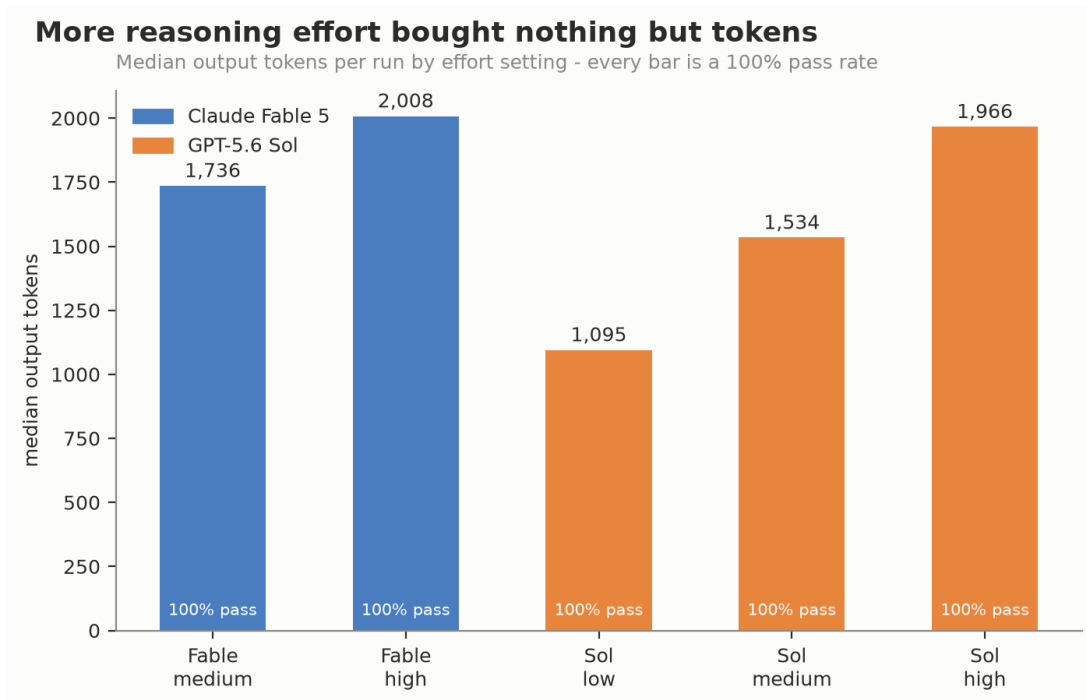


Figure 2: Output tokens by effort level. Tokens climb with the effort setting; every marker is a 100% pass. Effort buys tokens, not outcomes.

5.3 Different animals, same destination

The models’ solution styles differ sharply. Claude Fable 5 iterates: median 6–7 turns per task, ≈ 59 –68 s wall clock — read, edit, check, edit again. GPT-5.6 one-shots: median 1 turn, ≈ 26 –64 s — internal reasoning, then a single emitted solution. Neither style won on outcomes, because both cleared the tier. Turn count is a workflow signature, not a quality signal.

5.4 The harness did not move pass rate

Paired bare-vs-harnessed comparisons at each model’s winning effort (14 pairs each) again produced zero discordant pairs: 100% $\rightarrow$ 100%. The harness was designed for longer, messier work than these tasks; on this suite it changed token spend (Fable -560 mean tokens; Sol $+71,624$, driven by its instruction-context handling) but not outcomes.

5.5 Token economics: who is cheaper per solved task?

Table 3 reports the exact sign-flip permutation test (Eq. 3) on per-task median $\log(\text{total tokens})$ over passing runs. All nine paired differences favor Claude Fable 5; the observed sum of differences is -17.72 , and 2 of $2^9 = 512$ sign patterns are as or more extreme, giving two-sided $p = 0.0039$. By its own CLI’s accounting, Claude Fable 5 is roughly 6–10 $\times$ cheaper in total tokens per solved task on this suite.

task	Fable med log(tok)	Sol med log(tok)	diff (F–S)
t1-py-a	9.9293	11.7945	−1.8652
t1-py-b	9.9256	11.7996	−1.8739
t1-ts-a	9.9218	11.7104	−1.7886
t1-ts-b	9.9579	11.9564	−1.9985
t2-py-a	9.9333	11.6615	−1.7282
t2-py-b	9.9590	12.0185	−2.0595
t2-ts-a	10.0253	12.2987	−2.2734
t2-ts-b	9.9256	11.9755	−2.0499
t3-a	9.8712	11.9511	−2.0799
sum of diffs = -17.7172 ; 2 of 512 patterns as-or-more extreme $p = 0.0039$			

Table 3: Exact sign-flip permutation test on per-task median $\log(\text{total tokens})$ over passing runs, paired by task.

A caveat you cannot skip: input tokens do not compare across vendors. Claude Fable 5’s CLI reports $\approx 19.4\text{k}$ median input tokens; GPT-5.6’s reports 95k–175k. That is not GPT-5.6 reading 5–9 $\times$ more code: the two CLIs count context resends and cache reads differently, so their input-token numbers measure different things under the same label (Figure 3). Within-model comparisons are safe (each CLI is self-consistent); cross-model input-token or dollar comparisons are indicative only, and we do not publish them as hard claims. The honestly cross-comparable axes are output tokens, turns, and wall-clock time.

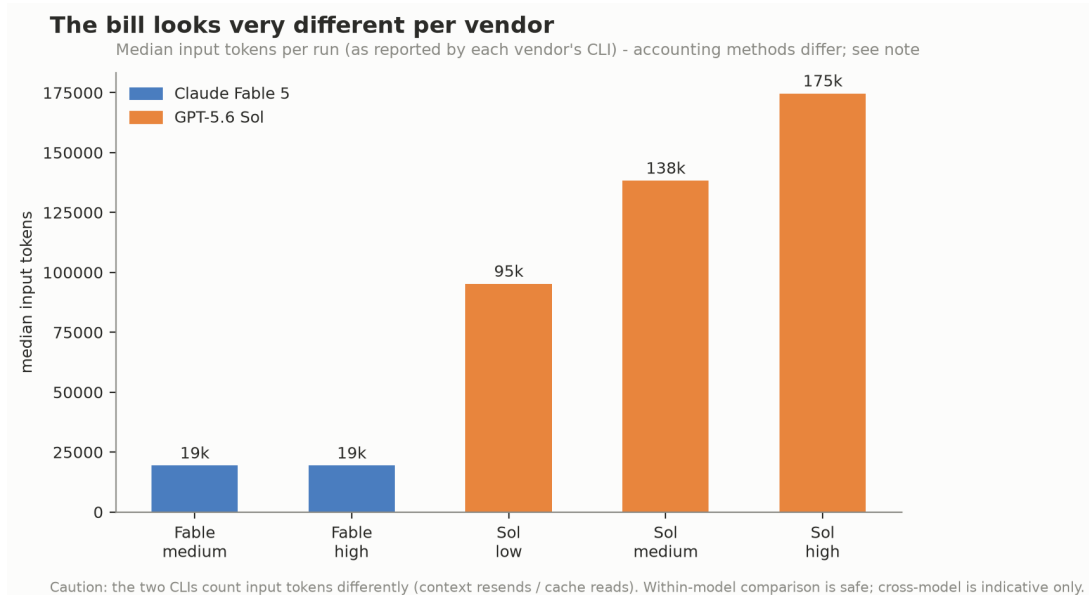


Figure 3: Input tokens by configuration. The cross-model gap is an artifact of CLI bookkeeping, not workload; the chart carries the caveat on its face.

5.6 The hybrid arm: full disclosure

We also tested a hybrid configuration — Claude Fable 5 orchestrating GPT-5.6 as the implementer — on the greenfield task. It was the only configuration that did not run clean. Across four attempts, two produced planning notes and zero code; two shipped working solutions of ≈ 160 –176 lines: **2 of 4**. The official results file records the two later, successful attempts; the two earlier failures are disclosed here. The rig was identical across all four attempts, so the most defensible reading is genuine run-to-run variance in a multi-agent handoff, not a fixed defect — and at $n = 4$ we decline to build any stronger story. It is a flag for the next experiment, not a conclusion.

5.7 Judge agreement

Across 139 dual-judged runs (each judge’s score averaged over its four rubric axes): mean absolute gap **0.53** points on a 0–10 scale, Pearson $r = 0.76$, and **96%** of runs within ± 1 point. On the pilot, the GPT judge scored ≈ 0.5 points more generous than the Claude judge on the same diffs, and both judges leaned slightly toward GPT-5.6’s code — precisely the self-preference risk that motivated blind, dual judging [4], and precisely why we report the disagreement instead of averaging it away.

6 Limitations

This section is the credibility of the study; we state each limit bluntly.

1. **Small per-arm samples.** At $n = 24$ per arm the design has 80% power only for gaps $\gtrsim 36.7$ percentage points at a 50% baseline (Eq. 4). Smaller real differences get a confidence interval, not a verdict.

2. **Ceiling effect.** Because both models passed everything, the suite cannot *rank* them — it only establishes that both clear this tier. Reading “154 perfect runs” as “the models are identical” over-reads the data.
3. **Model judges, not humans.** Blind, dual, disagreement reported — but still automated graders with their own biases.
4. **One re-run.** A single Claude Fable 5 greenfield run hit a transient CLI error (2.3s, zero tokens) and was re-run. Disclosed.
5. **AI-assisted task authoring.** Tasks were proven solvable, but their difficulty was miscalibrated low. That miscalibration is not a footnote; it *is* the headline finding.
6. **No dollar claims.** Cost figures use list-price placeholders over vendor-incomparable token counts and are not published as dollar claims.
7. **Long-context (“dumb zone”) asymmetry.** Practitioner experience and published long-context research find that model quality degrades as context grows large ($\sim 100k$ tokens) [5]. By its own CLI’s accounting, GPT-5.6 exceeded 100k cumulative input tokens on 65 of its 88 runs (median 142k, max 353k); Claude Fable 5 peaked at 19.6k. Two qualifiers: the `codex` CLI counts context resends across internal steps, so live context occupancy at any moment is not observable from our data; and the bias runs *against* GPT-5.6, which passed everything anyway. The tie is therefore safe, but blind-judge quality comparisons carry this asymmetry, and it is on the record.

7 Future work

A ceiling tells you exactly where to point the next experiment. Experiment 2 is a hard tier (T4): the same rig, blocking, blind dual judging, and exact statistics, with task difficulty raised until failures appear. Failures are the requirement — a gap becomes measurable only once at least one model starts missing, and the effort dial can only earn its cost once “think harder” changes an outcome rather than a token count. Same exam room, harder exam.

Acknowledgments

The experiment harness, task suite, dual-judge pipeline, and statistical appendix are maintained in a private repository; the statistical methodology is grounded in the author’s industrial-engineering coursework (Georgia Tech ISyE): hypothesis testing, confidence intervals, design of experiments, and stochastic processes.

A Reproducibility

Every number in this report is produced by a dependency-free script from the raw logs:

```
python3 runner/stats.py --results runner/results/results.jsonl
--judgments runner/results/judgments.jsonl
```

The script emits the statistical appendix (per-cell Wilson CIs, every McNemar 2×2 table with its exact p -value, the token permutation test, judge agreement, and the power statement) and carries a self-test mode (`--selftest`) checking the primitives against known values: McNemar exact $p(b=1, c=5) = 0.21875$; Wilson 95% CI for $8/10 = (0.4901, 0.9433)$; sign-flip p for eight positive differences $= 2/256 = 0.0078125$; Pearson $r = 1.0$ on exact linear data. All checks pass.

Judging is blind and dual: transcripts and diffs are stripped of identity before either judge scores them, and both a Claude judge and a GPT judge score every run on the same four-dimension rubric.

References

- [1] N. Jain, K. Han, A. Gu, W.-D. Li, F. Yan, T. Zhang, S. Wang, A. Solar-Lezama, K. Sen, and I. Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv:2403.07974*, 2024.
- [2] Y. Sui et al. Stop overthinking: A survey on efficient reasoning for large language models. *arXiv:2503.16419*, 2025.
- [3] A. Su et al. Between underthinking and overthinking: An empirical study of reasoning length and correctness in LLMs. *arXiv:2505.00127*, 2025.
- [4] K. Wataoka, T. Takahashi, and R. Ri. Self-preference bias in LLM-as-a-judge. *arXiv:2410.21819*, 2024.
- [5] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. *arXiv:2307.03172*.
- [6] E. B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927.
- [7] Q. McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.
- [8] R. A. Fisher. *The Design of Experiments*. Oliver and Boyd, Edinburgh, 1935.
- [9] D. C. Montgomery. *Design and Analysis of Experiments*. Wiley, 9th edition, 2017.